

Electric Taxi Platform

Microservices Backend Architecture

This document defines the backend architecture for the Electric Taxi Management System. The platform is composed of twelve independently deployable microservices behind a single API gateway, communicating synchronously over REST/gRPC for request–response operations and asynchronously over a central event bus for real-time ride, tracking, and geofence-violation events. Every service owns its data store exclusively.

VERSION	STATUS	DATE	OWNER	CLASSIFICATION
1.0	Approved for Build	July 2026	Platform Engineering	Internal

01 Architecture Principles

- Single entry point.** All client traffic terminates at the API gateway; no service is directly reachable from the internet.
- Database-per-service.** Each service owns an isolated data store. Cross-service access happens only through APIs or published events — never through another service's database.
- Sync where it must, async where it can.** Request–response calls use REST/gRPC; state changes are broadcast as domain events on the bus so consumers stay decoupled.
- Independent deployability.** Every service ships through its own CI/CD pipeline and can be scaled, versioned, and rolled back in isolation.
- Real-time first.** High-frequency GPS ingestion, live tracking, and dispatch matching are served from in-memory stores (Redis GEO) with durable history in time-series storage.

02 System Overview

Electric Taxi Management System – Backend Architecture · v1.0

iTechnologi Group · Confidential



03 API Gateway — Single Entry Point

A managed gateway (Kong, NGINX, or KrakenD) fronts every service and is the only component exposed publicly. It owns the following cross-cutting responsibilities:

Authentication – JWT verification

Request routing

Rate limiting

Load balancing

Request / response transformation

API versioning

WebSocket upgrade – live tracking

Twelve services grouped into four business domains. Each row lists the service's single responsibility and the data store it exclusively owns.

#	SERVICE	RESPONSIBILITY	DATA STORE
01	 Auth Service	Login, OTP, JWT issuance, role-based access for passenger / driver / admin.	auth_db (PostgreSQL)
02	 Passenger Service	Passenger profiles, saved payment methods, ride history.	passenger_db (PostgreSQL)
03	 Driver Service	Driver & vehicle onboarding, documents, online/offline status.	driver_db (PostgreSQL)
04	 Dispatch Service	3-nearest-zone broadcast matching; zone expansion when no driver is available.	Redis (in-memory state)
05	 Geofence Service	Zone CRUD, vehicle–zone pairing, boundary checks, violation detection.	geofence_db (PostGIS)
06	 Tracking Service	High-frequency GPS ingestion from vehicle trackers; live location cache.	Redis GEO + TimescaleDB
07	 Ride Service	Ride state machine: requested → assigned → in-progress → completed.	ride_db (PostgreSQL)
08	 Fare Service	Configurable fare-formula engine over final distance and time.	fare_db (PostgreSQL)
09	 Payment Service	Cash confirmation, JazzCash and card-gateway processing, receipts.	payment_db (PostgreSQL)
10	 Notification Service	Push (FCM) and SMS dispatch for ride / driver / admin events.	notif_db (PostgreSQL)
11	 Admin / Reporting	Dashboards, violation reports, fleet and revenue analytics.	Event-stream reads (CQRS)
12	 Document / Media	Driver license / CNIC uploads, ride receipts, object storage.	S3-compatible storage

 Identity & Users  Ride Operations  Money  Comms & Admin

05 Event Bus — Asynchronous Communication

Electric Taxi Management System — Backend Architecture · v1.0

iTechnologi Group · Confidential

A central broker (Kafka or RabbitMQ) carries domain events between services. Producers publish once; any service may subscribe without coupling to the producer. Core topics:

`ride.requested`

`ride.assigned`

`ride.started`

`ride.completed`

`vehicle.location.updated`

`geofence.violation.detected`

`payment.completed`

`driver.status.changed`

06 Data Layer

Database-per-Service (PostgreSQL)

Each microservice owns an isolated PostgreSQL database, with the PostGIS extension for Geofence and Tracking. No service reads another's database directly — access is only via its API or published events.

Redis

Live driver locations (GEO commands), session cache, and dispatch matching state held in memory for sub-millisecond reads.

TimescaleDB

Durable time-series history for high-frequency GPS pings, powering trip replay and analytics.

Object Storage (S3-compatible)

Driver documents (license, CNIC), ride receipts, and uploaded media in a versioned bucket.

07 External Integrations

JazzCash API

Mobile-wallet payment collection and confirmation callbacks.

Card Gateway

1LINK-compliant bank gateway for debit/credit card processing.

FCM + SMS Gateway

Push-notification and SMS delivery for ride and operational events.

GPS Tracker Hardware

In-vehicle trackers streaming positions over MQTT/GSM into the Tracking Service.

08 Cross-Cutting Infrastructure

Service Discovery

Consul or Kubernetes DNS for internal service resolution.

Centralized Logging

ELK / Loki stack aggregating structured logs from every service.

Monitoring & Alerting

Prometheus metrics with Grafana dashboards and alert routing.

CI/CD per Service

Docker images built and deployed through independent pipelines.